SYSTEM DESIGN INTERVIEW

Design YouTube

A global video streaming platform handling 2B+ users, 500 hours/minute uploads, 1B hours/day watched

~ 45 minute read

What's Inside

01 Problem Statement

Ask 7 clarifying questions

You are asked to design **YouTube**, a global video streaming platform where users can upload, watch, and share videos. The interviewer expects you to start with clarifying questions to scope the problem before sketching architecture.

Key Clarifying Questions

Question	Answer / Scope
1. Scale: how many users?	2 billion total users; ~500 million monthly active users
2. Video upload volume?	500 hours of video uploaded per minute globally
3. Video storage duration?	Indefinite (≥ 10 years); some pruning of very old unwatched content
4. Geographic reach?	Global; must support playback in all regions with local CDN
5. Real-time features?	Live streaming out of scope for MVP; focus on stored video on-demand
6. Video resolution?	Multiple: 144p (mobile) up to 4K; client-side device adaptivity
7. Core product focus?	Watch experience > upload experience. Search, recommendations, comments are secondary.

Design Constraints (from clarifications)

- **Global scale:** 500M MAU spread across 6 continents → geographically distributed CDN
- **Upload velocity:** 500 h/min = 30,000 h/day → massive ingestion pipeline
- **Watch volume:** ~1 billion video hours watched per day globally (estimated)
- **Read-heavy:** watch:upload ratio ≈ 2000:1; bias the architecture toward playback
- **Quality expectations:** <2 sec startup, near-zero buffering, instant pause/resume

02 Functional & Non-Functional Requirements

What the system must do, and how well

Functional Requirements

- **Upload video:** chunk-based resumable upload with progress tracking
- **Watch video:** stream with ABR (adaptive bitrate), low latency, seek support
- **List videos:** search by title/tags, browse by category, trending videos
- **Recommend videos:** personalised picks based on watch history & collaborative filtering
- **Comments:** post, reply, like; nested threads with pagination
- **Subscriptions:** subscribe to channels and get notifications for new uploads

Non-Functional Requirements

Metric	Target
Video startup latency	<2 seconds (p99)
Rebuffer rate	<0.5% of streams
Availability	99.99% (4 nines)
Search latency	<100ms (p99)
Recommendation latency	<500ms (p99)
Storage durability	99.999999999% (11 nines, triple replication)
Video encoding latency	<4 hours for 1 TB (≈ 2 Mbps processing)

03 Capacity Estimation

Calculate QPS, storage, bandwidth

Daily Upload Volume

```
500 hours/minute × 60 minutes = 30,000 hours/day

Assuming average video is 10 minutes:
30,000 h/day × 60 min/h ÷ 10 min/video = 180,000 videos/day

Upload QPS (spread over 24h = 86,400 sec):
180,000 / 86,400 ≈ 2.1 video_uploads/second average

Peak-hour ≈ 3× average = ~6–7 uploads/sec at peak
```

> **WARNING**
>
> **Don't mix units**
>
> An earlier draft wrote *30,000 hours = 30,000 × 60 min = 1,800,000 videos/day* — that multiplies hours by minutes-per-hour and labels the result as videos. The correct formula divides total minutes by minutes-per-video, giving **180,000 videos/day** (10× smaller).

Daily Storage Requirement

```
Raw video storage:
  30,000 hours/day × 1 GB/hour = 30 TB raw video/day

Transcoded (~10 variants: 4K + 1080p + 720p + 480p + 360p
+ 240p + 144p, each in H.264 / VP9):
  30 TB × ~10 transcoded formats ≈ 300 TB/day

Annual storage needed:
  300 TB/day × 365 = 109.5 PB/year

With 3× replication for durability:
  109.5 PB × 3 ≈ 328 PB/year of replicated storage
```

- **Raw daily ingest:** 30 TB/day at 1 GB per hour of source video
- **After transcoding:** ~300 TB/day across the resolution & codec ladder
- **Annual replicated:** 109.5 PB/year × 3 ≈ **328 PB** per year of stored content
- **5-year retention:** ~547 PB logical / ~1.6 EB replicated, assuming linear growth

Streaming (Watch) Volume & Bandwidth

```
Estimated daily watch hours: ~1 billion hours/day

Concurrent streams (uniform across the day):
  1B hours/day ÷ 24 h/day ≈ 41.7 million concurrent viewers

Bandwidth requirement (at 2 Mbps average adaptive bitrate):
  41.7M streams × 2 Mbps ≈ 83 Tbps average egress

Peak ≈ 2× average (diurnal / regional skew):
  ~170 Tbps CDN egress at peak

With 95% edge cache hit rate, origin pull:
  5% × 83 Tbps ≈ 4 Tbps to origin
```

WARNING

Hours per day, not seconds per day

An earlier draft divided *1 B hours/day by 86,400 seconds/day*, mixing hours with seconds and arriving at 11.6M concurrent viewers. The correct denominator is 24 hours/day, giving **≈ 41.7M concurrent viewers**. Every downstream bandwidth number (egress, peak, origin pull) is ~3.6× larger as a result.

Database Metrics

- **Videos table:** ~660M total over 10 years (180K/day × 365 × 10); ~500 B/row ≈ 330 GB
- **Users table:** ~2 billion users; ~1 KB/row ≈ 2 TB
- **Comments:** ~5 B comments (assume 7:1 comment:video ratio); Cassandra sharded by video_id
- **Likes / views:** ~50 B+ events; Redis counters with async flush to Cassandra

KEY

Key Numbers

Uploads: **180K videos/day** (~2 QPS avg, ~6–7 peak) · Raw ingest: **30 TB/day** · Transcoded: **~300 TB/day → 328 PB/yr** (3×) · Concurrent viewers: **~41.7M** · CDN egress: **~83 Tbps** avg / **~170 Tbps** peak · Origin pull: **~4 Tbps**.

04 High-Level Architecture

Multi-tier service design

The system is divided into several layers: **client**, **edge / CDN**, **API gateway**, **core services**, and **data tier**. Each layer is independently scalable and fault-tolerant. The key split is **write path** (upload → transcode) versus **read path** (stream from CDN); they scale on entirely different curves.

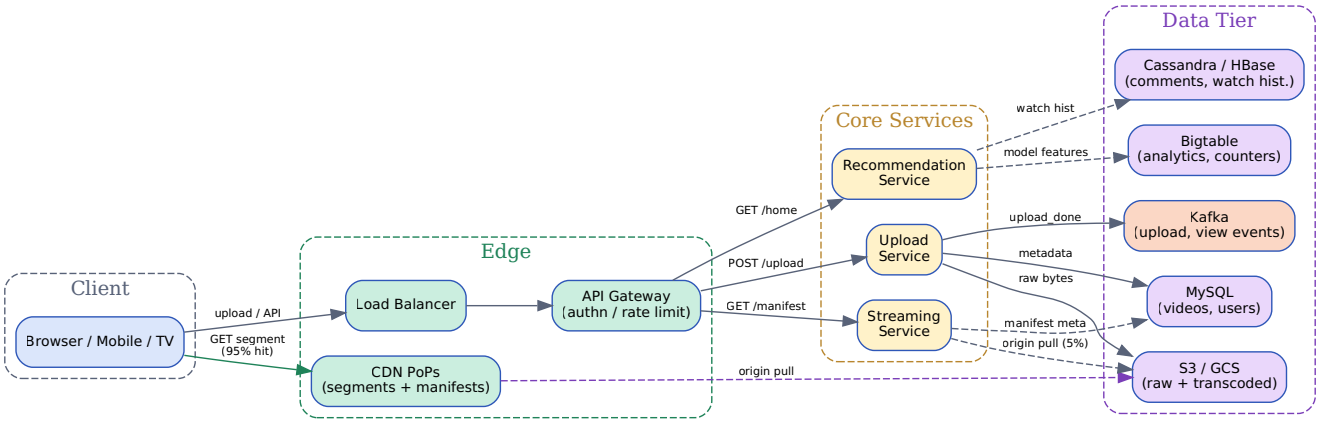

Figure 4.1 — Viewer and creator traffic flow through distinct paths so each side can scale independently.

Component Overview

- **Client:** Web browser, mobile app, smart TV; initiates playback or upload
- **Load Balancer:** distributes inbound requests across API gateways
- **API Gateway:** authentication, rate limiting, request routing
- **Upload Service:** chunked / resumable uploads (TUS protocol)
- **Streaming Service:** serves HLS / DASH manifests and segments
- **Search Service:** Elasticsearch for title / tag search with ranking
- **Recommendation Service:** two-tower model (candidates → ranking)
- **Transcoding Cluster:** parallel workers encoding videos into the resolution × codec ladder
- **Message Queue (Kafka):** event-driven pipeline; `upload_done` → `transcode_trigger`
- **S3 / GCS:** raw video storage (ingest) and transcoded segments (streaming)
- **CDN PoPs:** geographically distributed Points-of-Presence caching segments
- **MySQL:** transactional metadata (videos, users)
- **Cassandra / HBase:** time-series comments & watch history; scales horizontally
- **Bigtable:** view counts, model features, analytics aggregations
- **Redis:** caching, session store, real-time counters (likes, views)

INSIGHT

Read path vs write path

The architecture explicitly separates the write path (*upload → transcode → object store*) from the read path (*CDN → streaming service → object store*). Uploads happen at single-digit QPS; reads happen at tens of millions of concurrent streams. Decoupling lets each side scale on its own clock.

05 Video Upload & Ingestion

Resumable uploads, deduplication

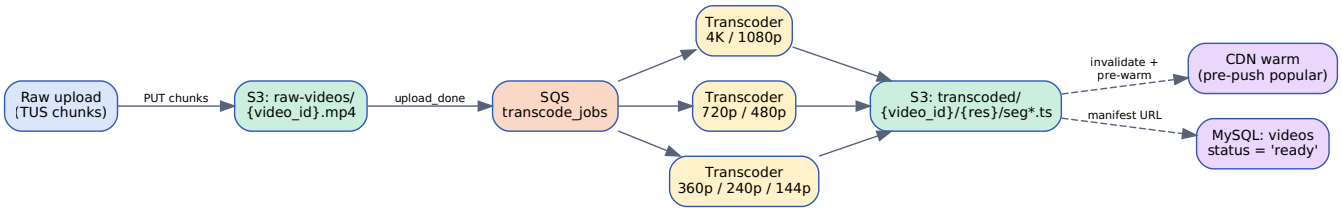

Figure 5.1 — Transcoding pipeline. Raw bytes land in S3, an SQS message fans out to a worker pool, each variant is written back to S3 and the CDN is warmed.

TUS Protocol (Resumable Upload)

The Tus protocol (`https://tus.io/`) standardises resumable uploads. YouTube-scale requires:

- **Chunking:** split video into 5–10 MB chunks (standard chunk size)
- **Resumability:** track which chunks uploaded; client resumes from last chunk on network failure
- **Server acknowledgment:** Upload Service returns offset and next chunk index
- **Atomic commits:** mark upload complete only when all chunks received and checksummed

Upload Flow (Step-by-Step)

1. Creator initiates upload: `POST /upload` → receives `upload_id`, chunk size hints
2. Client chunks & sends: `PATCH /upload/{id}/chunk/{n}` with chunk + checksum
3. Server acknowledges: returns offset progress; client buffers retries on transient error
4. Upload completes: final chunk triggers verification; checksum mismatch → re-upload chunk
5. Kafka event published: `upload_complete {video_id, creator_id, original_size}`
6. Transcoding orchestrator consumes the event and dispatches tasks to transcoding workers
7. Raw video stored: S3 at `s3://raw-videos/{creator_id}/{video_id}.mp4`

Deduplication & Content-Addressed Storage

- **Perceptual hashing:** compute pHash of uploaded video during transcoding
- **pHash lookup:** check if hash exists in Cassandra; if match → link `video_id` to existing content
- **Space savings:** avoid 50+ TB/day duplicate storage (estimated 10–15% duplicate rate)
- **Copyright enforcement:** identical pHashes flag potential copyright violations for review

Handling Upload Failures

- **Transient network failure:** client retries chunk with exponential backoff
- **Timeout on chunk:** Upload Service holds state; client resumes after 5 min
- **Corrupted chunk (checksum mismatch):** client re-sends; server re-computes checksum
- **Upload abandoned:** garbage collector cleans incomplete uploads after 7 days

> **WARNING**
>
> **Abuse mitigation**
>
> YouTube accepts uploads from anyone. Initial uploads land in a **moderation queue** for bot detection (content-hash lookup, NSFW classifier, copyright fingerprint). Cleared uploads then proceed to the transcoding pipeline.

06 Transcoding

Codec selection, resolution ladder, cost

Codec Comparison

Codec	Bitrate (1080p)	Compression	Adoption	Notes
H.264 (AVC)	3–5 Mbps	Good (2003)	Universal (99.9%)	Highest compatibility; patent-pool licences
VP9	2–3 Mbps	Excellent	~80% devices	Open-source; better compression than H.264
AV1	1–1.5 Mbps	Excellent+	Emerging (~40%)	Next-gen; high CPU cost; 5–10× encode time
HEVC (H.265)	2–3 Mbps	Excellent	~75% devices	Patent-heavy licensing; superior quality at bitrate

YouTube's Strategy

- **Primary:** H.264 — universal fallback for all browsers
- **Secondary:** VP9 — Chrome, Firefox; saves ~30% bandwidth
- **Tertiary:** HEVC — Apple devices (iOS / macOS); saves ~40% vs H.264
- **Future:** AV1 for premium content (4K+); encoding cost too high for bulk

Resolution Ladder

```
Standard ladder (~10 resolutions × 2–3 codecs):

  4K (2160p): H.264 (8–12 Mbps) | VP9 (5–7 Mbps) | HEVC (4–6 Mbps)
  1440p:      H.264 (5–8 Mbps)  | VP9 (3–5 Mbps) | HEVC (2.5–4 Mbps)
  1080p:      H.264 (3–5 Mbps)  | VP9 (2–3 Mbps) | HEVC (1.5–2.5 Mbps)
  720p:       H.264 (2–3 Mbps)  | VP9 (1–2 Mbps) | HEVC (1–1.5 Mbps)
  480p:       H.264 (1–1.5 Mbps)| VP9 (0.8–1.2 Mbps)
  360p / 240p / 144p: H.264 only (mobile-optimised, lowest bitrate)
```

Transcoding Cost & Performance

- **CPU-intensive:** 1 hour of 1080p H.264 encoding ≈ 45 min CPU on an 8-core machine

- **Parallel strategy:** 8 workers per node; each encodes a different resolution simultaneously
- **Cost optimisation:** ~$0.015 per hour of video transcoded (compute + storage + bandwidth)
- **SLA:** >90% of uploads transcoded within 4 hours; <10% within 24 hours

TIP

Why parallelise per-resolution

Each variant is independent — different resolutions read the same source but produce disjoint outputs. Fan out to workers per (resolution × codec) pair; the highest-resolution rung dominates wall-clock time, but the smaller rungs finish quickly and start hitting the CDN sooner.

07 Adaptive Bitrate Streaming

ABR algorithms, manifest format

HLS (HTTP Live Streaming) Manifest

```
#EXTM3U
#EXT-X-STREAM-INF:BANDWIDTH=2500000,RESOLUTION=1280x720
720p/segment_list.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=1500000,RESOLUTION=854x480
480p/segment_list.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=500000,RESOLUTION=426x240
240p/segment_list.m3u8

# master.m3u8 lists all available quality levels;
# each child playlist lists the .ts segments for its rung.
```

Adaptive Bitrate (ABR) Algorithms

Algorithm	Strategy	Latency	Fairness
BOLA (Buffer-based)	Maximise quality while maintaining buffer	Low	High
Pensieve (ML-based)	RL model predicts optimal bitrate	Very low	Very high
Rate-based	Adjust bitrate based on network probe	Medium	Medium
Buffer-filling	Ramp up quality until buffer > threshold	High	Low

BOLA (Buffer-Optimal Latency-Aware)

BOLA uses a utility function to maximise quality while avoiding stalls. The decision rule per segment:

```
Q(r) = log(bitrate[r]) − stall_penalty × P(stall | buffer, rate)

Decision: select the highest Q(r) such that buffering won't occur.
Re-evaluate every 1–3 segments based on (buffer level, throughput).
```

- **Provably optimal** in the offline (Lyapunov) setting
- **Stable:** rarely switches bitrate (reduces audio / visual artefacts)
- **Fairness:** balances competing streams sharing the same bottleneck
- **Buffer-aware:** escalates quality only when buffer is healthy

Manifest Request Flow

1. Player requests manifest: `GET /manifest?video_id=abc&device=mobile`
2. Streaming Service resolves: Redis cache hit (~99%); returns cached `.m3u8`
3. Cache miss (< 1%): hits MySQL → retrieves list of quality tiers + segment URLs
4. Manifest sent to player: includes `master.m3u8` with bandwidth hints
5. Player selects quality: BOLA picks the highest sustainable rung
6. Segment fetch: `GET /segments/720p_seg001.ts` → CDN PoP (~95% hit rate)
7. Adaptive switch: every 1–3 segments, player re-evaluates buffer + bandwidth

> **INSIGHT**
>
> **DASH vs HLS**
>
> DASH (Dynamic Adaptive Streaming over HTTP) is the codec-agnostic ISO standard; HLS is Apple's manifest format and dominates on iOS / Safari. Both use the same primitives — fragmented MP4 (or MPEG-TS), a master manifest, and per-rung playlists — so a single transcoded library can serve both with minimal duplication.

08 CDN Architecture

Multi-tier caching, geo-distribution

CDN Tier Strategy

- **Tier 1 (Edge PoPs):** closest to users; cache popular content (hot videos)
- **Tier 2 (Regional):** larger caches; fallback for Tier 1 misses
- **Tier 3 (Origin):** primary S3 / object storage; holds all transcoded segments

Cache Optimisation

- **Hit-rate targets:** 95–99% at edge; 99.5%+ at regional
- **Popular content:** pre-warm Tier 1 for trending videos (auto-detected by analytics)
- **TTL strategy:** long-lived content (> 30 days) → TTL 365d; new content → TTL 7d
- **Purge on update:** if creator re-uploads thumbnail / metadata, invalidate cache entries

Geo-Distribution

```
Estimated PoP locations (simplified):

  North America:  ~50 PoPs (NYC, LA, Chicago, Denver, …)
  Europe:         ~40 PoPs (London, Paris, Frankfurt, Moscow, …)
  Asia-Pacific:   ~60 PoPs (Tokyo, Singapore, Sydney, Delhi, …)
  South America:  ~15 PoPs (São Paulo, Buenos Aires, …)
  Africa:         ~10 PoPs (Johannesburg, Cairo, …)

  Total: ~175 PoPs globally
```

Traffic Flow & Load Balancing

- **DNS Anycast:** user resolves `video.youtube.com` → closest PoP via GeoDNS
- **HTTP redirect:** if PoP is full / low-capacity, redirect to next-nearest PoP
- **Load balancing:** round-robin across PoP servers; monitor CPU and bandwidth per server

> **KEY**
>
> **Why 95% edge hit rate matters**
>
> At ~83 Tbps average egress, every percentage point of cache miss is ~830 Gbps of extra traffic to origin. Pushing edge hit rate from 90% to 95% halves the origin pull from ~8 Tbps to ~4 Tbps — that's the difference between needing one origin region and three.

09 Database Design

Schema, sharding, consistency

Primary Databases

- **MySQL (Videos, Users):** strongly consistent; transactional; ACID for critical data
- **Cassandra (Comments, Watch History):** eventually consistent; high write throughput; distributed sharding
- **Redis (Cache, Counters):** in-memory; sub-millisecond latency; volatile but acceptable for ephemeral data
- **Elasticsearch (Search Index):** full-text indexing; ranked search; refreshed every 30 seconds

MySQL Schema (Simplified)

```
CREATE TABLE videos (
  video_id      BIGINT PRIMARY KEY,
  creator_id    BIGINT NOT NULL,
  title         VARCHAR(500) NOT NULL,
  description   TEXT,
  duration      INT,                      -- seconds
  created_at    TIMESTAMP,
  updated_at    TIMESTAMP,
  status        ENUM('processing','ready','blocked'),
  manifest_url  VARCHAR(255),              -- s3 origin
  thumbnail_url VARCHAR(255),
  view_count    BIGINT DEFAULT 0,
  like_count    BIGINT DEFAULT 0,
  INDEX idx_creator (creator_id),
  INDEX idx_status  (status)
);
```

Cassandra Schema (Watch History)

```
CREATE TABLE watch_history_by_user (
  user_id          BIGINT,
  watched_at       BIGINT,    -- timestamp in millis (sorted DESC)
  video_id         BIGINT,
  watched_duration INT,
  quality          VARCHAR(10),
  PRIMARY KEY (user_id, watched_at)
) WITH CLUSTERING ORDER BY (watched_at DESC);
```

```
-- Distributed: sharded by user_id
```

Sharding Strategy

- **MySQL (videos):** shard by `video_id` hash; ~10–20 shards (growth headroom)
- **Cassandra (comments):** shard by `video_id`; each node replicates 3 copies
- **Consistency:** MySQL = strong; Cassandra = eventual (acceptable for comments)

10 Video Search

Elasticsearch, ranking, autocomplete

Full-Text Search with Elasticsearch

- **Index:** video metadata (title, description, tags, category)
- **Indexing latency:** ~30 seconds (near-real-time); updated via Kafka topic
- **Sharding:** 10 primary shards + 2 replicas per shard
- **Typical query:** `'python programming'` → ranked by relevance, upload date, popularity

Search Ranking Formula

```
score(video) =
    TF-IDF(query, title)        × 0.40
  + TF-IDF(query, description)  × 0.20
  + BM25 (query, tags)          × 0.20
  + popularity_score(view_count, likes) × 0.15
  + recency_boost(1 - age_in_days / 365)

# Results sorted by score DESC, with a freshness boost for recent videos.
```

Autocomplete / Suggestions

- **Data source:** top 1M search queries (aggregated from analytics)
- **Prefix tree (trie):** stores common query prefixes with hit counts
- **Latency:** <100 ms (p99); served from Redis + local cache
- **Updates:** rebuilt hourly from query logs

11 Recommendation Engine

Two-tower, collaborative filtering

Two-Tower Architecture

- **Tower 1 (Candidate Generation):** fast, coarse-grained; retrieves ~1000 candidate videos
- **Tower 2 (Ranking):** slow, fine-grained; re-ranks the top 100 with a deep model
- **Advantage:** separates scale (Tower 1) from quality (Tower 2)

Candidate Generation (Tower 1)

- **Method:** collaborative filtering (user → user or item → item similarity)
- **Implementation:** user-video embedding space (Word2Vec-style on watch history)
- **Lookup:** nearest-neighbour search in embedding space (Faiss / ScaNN ANN library)
- **Latency:** <50 ms (cached results)

Ranking (Tower 2)

- **Model:** deep neural network (user context + video features → CTR probability)
- **Features:** watch history, user profile, video popularity, recency, seasonality
- **Output:** click-through rate (CTR) prediction; rank by expected CTR
- **Latency:** <200 ms for 100 videos via batch scoring

> **TIP**
>
> **Latency budget**
>
> Recommendation latency target: **<500 ms (p99)**. Trade-off: faster = fewer candidates, lower quality. The two-tower split buys you both — Tower 1 prunes 10^9 items down to ~1000 cheaply, Tower 2 spends its budget on the survivors.

12 Comments & Engagement

Threading, reply-to, pagination

Comment Storage Architecture

- **Database:** Cassandra (distributed, write-optimised)
- **Sharding:** by `video_id`; allows efficient range queries (recent comments first)
- **Denormalisation:** store comment + author info in one row (avoid joins)
- **Replication:** 3 copies; quorum reads (2) + writes (2) for consistency

Comment Thread Model

```
Top-level comment row:
  comment_id   : unique
  video_id     : sharding key
  author_id    : who wrote
  text         : comment body
  created_at   : timestamp
  reply_count  : cached counter
  like_count   : cached counter

Replies stored in a separate table:
  reply_id
  parent_comment_id  : foreign key
  author_id, text, created_at, like_count
```

Pagination & Sorting

- **Top comments:** sorted by `like_count` (cached in Redis); refreshed every 5 min
- **Recent comments:** sorted by `created_at DESC` (native Cassandra ordering)
- **Cursor pagination:** use `(last_comment_id, limit)` for stability
- **Latency:** <100 ms for 20 comments per page

13 Caching Strategy

Multi-layer cache hierarchy

Cache Layers

Layer	Technology	TTL	Hit Rate	Use Case
L1 Browser	LocalStorage / IndexedDB	1–7 days	90%+	Manifest, segment cache
L2 CDN Edge	Memcached / in-memory	1–7 days	95%+	Popular segment cache
L3 Regional	Memcached cluster	1–7 days	95%+	Fallback for edge miss
L4 Application	Redis (in-process)	1 hour	98%+	Metadata, session, counters
L5 Database	MySQL / Cassandra	N/A	N/A	Source of truth

Cache Invalidation Strategy

- **TTL-based:** most common; safe but may serve stale data
- **Event-driven:** publish to Kafka when metadata updated; subscribers invalidate
- **Hybrid:** short TTL (5–10 min) + event-based invalidation for critical data

WARNING

Cache stampede

If a popular item expires, many requests hit the database simultaneously. Mitigation: **probabilistic early expiration** (refresh before TTL with probability that grows as TTL approaches) or **distributed locking** so only one replica refills the entry.

14 Failure Modes & Resilience

Degradation, fallbacks, SLO targets

Key Failure Scenarios

- **Database shard down:** replica takes over (RTO <10s); alert ops team
- **CDN PoP offline:** Geo-DNS reroutes traffic to next-nearest PoP; users see <200 ms latency bump
- **Transcoding cluster overloaded:** queue uploads; SLA slips to 8–12 hours (acceptable)
- **Streaming Service degradation:** fall back to origin pull (slower but available); CDN load increases
- **Search outage:** disable search UI; show trending videos instead
- **Recommendation model stale:** use fallback heuristic (popular-by-category)

Circuit Breaker Pattern

```
if (search_service.failures > 10 in 30s) {
    circuit_breaker.trip()
    return fallback_heuristic()   # popular videos
    alert("Search service unhealthy")
}

// Retry with exponential backoff
for (attempt = 1 to 5) {
    wait(100ms * 2^attempt)
    if (search_service.health_check()) {
        circuit_breaker.reset()
        break
    }
}
```

SLO Targets

Service	Latency (p99)	Availability
Streaming API	<100 ms	99.99%
Upload Service	<500 ms	99.9%
Search	<150 ms	99.9%
Recommendations	<500 ms	99.95%

Service	Latency (p99)	Availability
CDN Segment Fetch	<50 ms	99.99%

15 Trade-Offs & Design Decisions

Decisions and rationale

Decision	Choice	Trade-off
Consistency model	MySQL (strong) for videos / users; Cassandra (eventual) for comments	Video metadata must be consistent (avoid serving mismatched manifests). Comments tolerate eventual consistency; MySQL is slower to write but guarantees correctness.
Encoding	Asynchronous (queue-based) transcoding	Encoding 1 TB takes 4 hours; blocking the user on a sync encode is unacceptable. Users see a 'processing' state and can watch a lower rung once the first variant finishes.
Transcoding topology	Distributed (workers in multiple regions)	Reduces latency for CDN warm-up; isolates regional failures; load-balanced across geographies. Cost is operational complexity (regional coordination, data transfer) — worth it at scale.
Codec strategy	Multi-codec (H.264 primary, VP9 secondary, HEVC for Apple)	Device compatibility plus bandwidth savings (VP9 ~30% less, HEVC ~40% less). Cost is 3× storage & encoding — justified by bandwidth savings on popular content.
Read / write split	CDN + Streaming Service for reads; Upload + Transcode for writes	Lets each side scale independently. Tens of millions of concurrent viewers vs. single-digit upload QPS — sharing infrastructure would force one side to over-provision.
Comment ordering	Top comments cached (Redis, 5 min); recent comments native Cassandra order	Two read patterns, two indices. Top comments are read-heavy and tolerate 5-min staleness; recent comments need real-time freshness.

16 Interview Playbook

How to ace the discussion

Opening (First 5 Minutes)

1. **Clarify scope:** 'Are we designing upload, playback, or both? What about live?'
2. **Estimate scale:** '2B users, 500 hours/min upload, 1B hours/day watched.'
3. **State assumptions:** 'I'll focus on on-demand playback; we can discuss live as a follow-up.'

Architecture Design (Next 15–20 Minutes)

1. Draw high-level diagram: `client → CDN → API → services → databases`
2. Separate **read path** (CDN + streaming) from **write path** (upload + transcode)
3. Explain the core trade-off: write-once, read-many → optimise for playback latency
4. Name key components: API gateway, upload service, streaming service, transcoding, message queue
5. Database choices: MySQL for metadata (consistency), Cassandra for comments (scale)

Deep Dives (Next 15 Minutes — Interviewer's Choice)

- **Upload:** TUS protocol, chunking, resumability, deduplication
- **Streaming:** HLS / DASH, ABR algorithms (BOLA), CDN cache hierarchy
- **Transcoding:** codec comparison, resolution ladder, cost optimisation, parallel workers
- **Search:** Elasticsearch, ranking formula, autocomplete, prefix trie
- **Recommendations:** two-tower model, candidate generation, ranking model, CTR prediction

Resilience & Trade-Offs (Final 5–10 Minutes)

1. Discuss failure modes: CDN down, database shard down, transcoding overload
2. Explain fallback strategies: circuit breakers, degraded modes, SLO-based prioritisation
3. Compare trade-offs: strong vs eventual consistency, sync vs async, multi-codec vs single

Common Pitfalls

- **Not clarifying scope:** treating live and on-demand the same (very different!)
- **Overcomplicating early:** jumping to database sharding before discussing API design
- **Ignoring the CDN:** assuming all content from origin misses ~90% of the optimisation
- **Single codec:** ignoring the bandwidth savings from VP9 / HEVC
- **No monitoring:** failing to mention SLOs, metrics, or alerting

- **Unit confusion:** mixing hours-per-day with seconds-per-day, or hours with minutes — always carry units through the math (the corrected concurrent-viewer and upload figures in §03 demonstrate why)

TIP

End strong

Mention one recent innovation (e.g., 'I'd explore AV1 for premium content if encoding cost improves') or a metric you'd track (e.g., 'I'd monitor cache hit ratio by region to optimise CDN warm-up').

KEY

Memorise These Numbers

Uploads: **180K videos/day** · Raw ingest: **30 TB/day** · Transcoded: **~300 TB/day → 328 PB/yr** (3×) · Concurrent viewers: **~41.7M** · CDN egress: **~83 Tbps** avg / **~170 Tbps** peak · Origin pull: **~4 Tbps** · Edge hit rate: **95%+**.